

Machine Learning — Assignment 2

k-Means · Gaussian Naïve Bayes · C4.5 Decision Tree

Student Name	Hafiz Abdullah Muhammad
Roll Number	BSAI23034
Random Seed	034 (last 3 digits of roll number)
Course	Machine Learning
Assignment	Assignment 2

Implementation Constraints

- **All algorithms implemented from scratch:** only NumPy, Pandas, Matplotlib used
- **scikit-learn permitted only for:** train_test_split and evaluation metrics
- **Decision Tree algorithm:** C4.5 (Gain Ratio) — NOT ID3
- **Datasets:** All generated programmatically (no Kaggle/public datasets)
- **Random seed:** 034 throughout for full reproducibility

Contents

Section	Content
Dataset Construction	Programmatic generation, seed=034
Question 1 — k-Means Clustering [30 marks]	Parts A, B, C
Question 2 — Gaussian Naïve Bayes [30 marks]	Parts A, B, C, D
Question 3 — C4.5 Decision Tree [40 marks]	Parts A, B, C, D, E
Summary and Conclusions	Cross-algorithm comparison

Dataset Construction

All datasets are generated programmatically using `numpy.random.default_rng(seed=34)`. No public or Kaggle datasets are used. Every dataset satisfies $n \geq 1000$ samples and $d \geq 15$ features (with at least 5 informative and 5 noisy features).

Dataset	n	d	Classes	Key Characteristic
Low-Noise	1,200	20 (10+10)	2	Spherical Gaussian blobs, well-separated
High-Noise	1,500	20 (5+15)	2	Overlapping classes + 15% label noise
High-Dimensional	5,000	60 (10+50)	2	XOR-like boundary, 50 noise dims $\leftarrow d \geq 50$ & $n \geq 5000$
k-Means Friendly	2,000	15 (10+5)	4	Spherical, balanced, well-separated clusters
k-Means Adversarial	1,000	4	2	Elongated ellipses + concentric rings
NB Correlated	2,000	10	2	$X_2 \approx X_1$ (corr $\approx$ 0.999), independence violated
NB Success	2,000	15	2	Correlated but symmetric across classes
NB Failure (XOR)	2,000	15	2	XOR boundary — NB completely fails
DT Greedy Example	1,200	15 (2+13)	2	Feature A (wide) vs Feature B (true signal)
DT Noise Sensitive	1,500	18 (8+10)	2	Clean base; noise introduced in experiment

Dataset Diversity Requirements — All Met

- **Low-noise dataset:** clean Gaussian blobs (algorithms succeed)
- **High-noise dataset:** overlapping classes + 15% label flips (algorithms struggle)
- **High-dimensional dataset:** $n=5,000$, $d=60$ with 50 noise features (satisfies BOTH $d \geq 50$ and $n \geq 5000$)

Question 1: k-Means Clustering [30 Marks]

Part A: Core Implementation

The KMeans class is implemented entirely from scratch using NumPy. All distance computations and cluster assignments are fully vectorized — no sample-wise Python loops. Two initialization strategies are supported: random and k-Means++.

Key Implementation Details

- **Distance computation:** Broadcasting $(n,1,d) - (1,k,d) \rightarrow (n,k,d) \rightarrow \text{sum} \rightarrow (n,k)$ squared Euclidean distances
- **Cluster assignment:** `np.argmin(distances, axis=1)` — single vectorized operation
- **Centroid update:** `np.add.at` accumulation with empty-cluster guard (reinitialize to random point)
- **Convergence criterion:** max centroid shift $< \text{tol}=1\text{e-}4$ (stops when centroids stabilize)
- **k-Means++:** Each new centroid selected proportional to D^2 distance from existing centroids

Results — Friendly Dataset (n=2000, d=15, k=4)

Initialization	Final WCSS	Iterations	Notes
k-Means++	66,530.00	22	Better starting positions, lower WCSS
Random	67,318.60	11	Faster convergence but worse local minimum

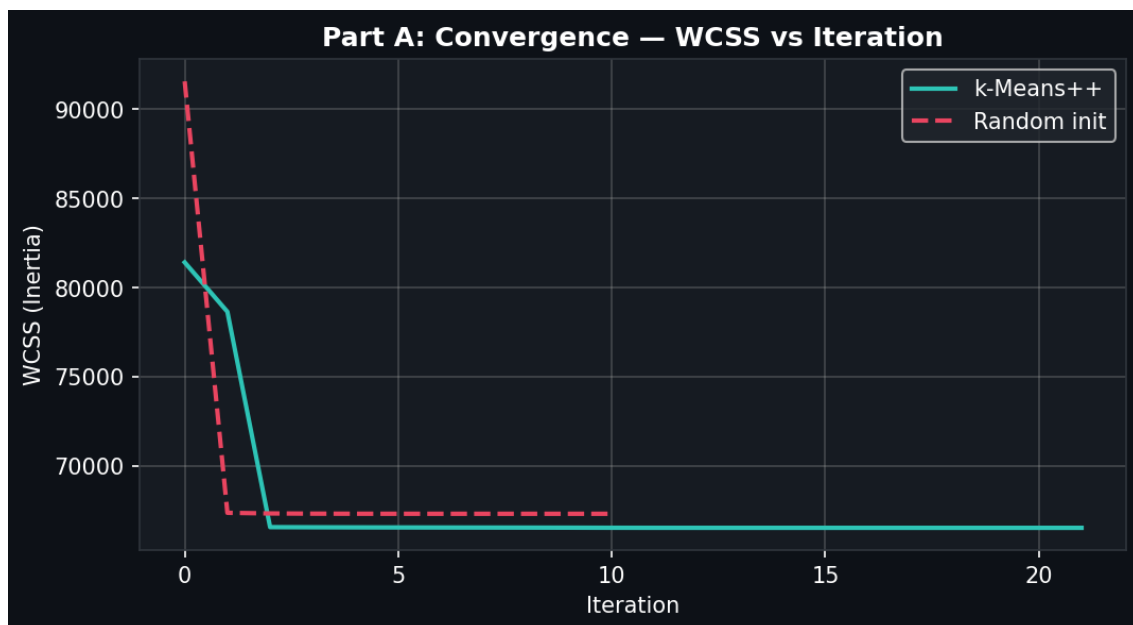


Figure 1.1: WCSS vs iteration — k-Means++ vs random initialization convergence curves

Part B: Adversarial Dataset Construction

Two datasets demonstrate k-Means' failure modes. The friendly dataset satisfies all k-Means assumptions. The adversarial dataset deliberately violates them with elongated ellipses and concentric rings.

Dataset	Cluster Accuracy	WCSS	Assumption Violated
Friendly (spherical blobs)	0.626	66,530	None — all assumptions satisfied
Adversarial — raw	0.787	30,888	Spherical shape (ellipses + rings)

Adversarial — scaled	0.518	3,214	Scaling cannot fix non-convexity
----------------------	-------	-------	----------------------------------

Why k-Means Fails on the Adversarial Dataset

- **Elongated ellipses:** k-Means minimizes Euclidean distance equally in all directions. Anisotropic clusters have different variances per axis — centroids are pulled to incorrect positions.
- **Concentric rings:** k-Means assumes convex cluster boundaries. Ring-shaped clusters are non-convex — inner/outer ring samples are equidistant from each ring's centroid, making correct assignment impossible.
- **Feature scaling:** Z-score normalization only helps when clusters differ in scale. It cannot fix violations of spherical shape or convexity assumptions.

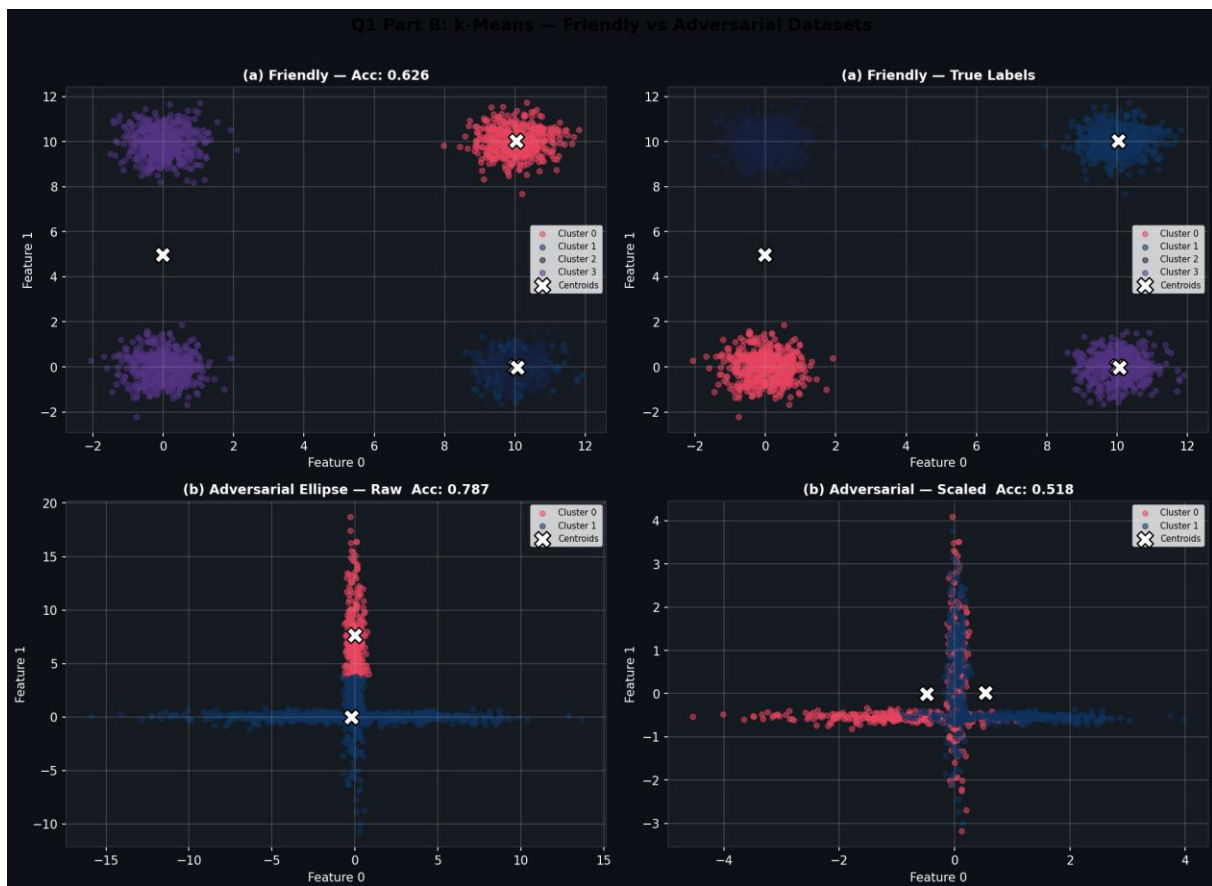


Figure 1.2: Friendly vs Adversarial datasets — cluster assignments, true labels, raw vs scaled comparison

Part C: Initialization Sensitivity (20 Runs)

k-Means was run 20 times with different random seeds on the friendly dataset (n=2000, d=15, k=4). Results demonstrate sensitivity to initial centroid placement.

Metric	Value
Best WCSS (Run 2)	17,214.28
Worst WCSS (Run 13)	116,868.95
Ratio (worst / best)	6.79×
Standard deviation	27,781.88

Why Different Initializations Lead to Different Local Minima

- **Basin of attraction:** Random initialization may place two centroids inside the same true cluster, leaving another cluster unrepresented. The algorithm cannot escape this poor start.
- **Non-convex objective:** The WCSS surface has many saddle points and local minima. Gradient-free optimization cannot guarantee global optimality.
- **Remedy: k-Means++:** Selects centroids proportional to D^2 distance from existing ones, spreading them across the space and dramatically reducing WCSS variance across runs.

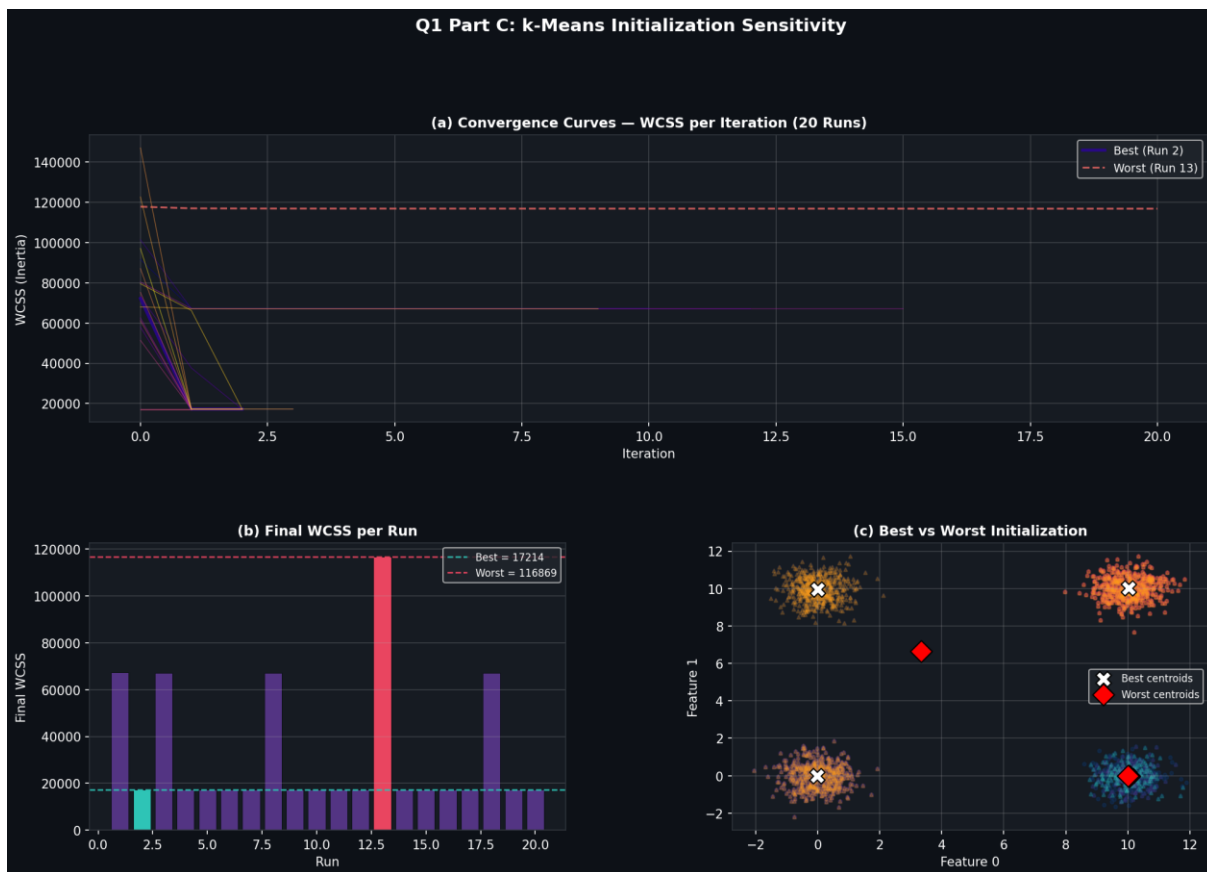


Figure 1.3: Convergence curves (all 20 runs), final WCSS per run, and best vs worst initialization cluster comparison

Question 2: Gaussian Naïve Bayes [30 Marks]

Part A: Implementation

The GaussianNaiveBayes class is implemented from scratch. All predictions use log-probabilities to avoid numerical underflow, with the log-sum-exp trick applied when converting to class probabilities.

Mathematical Foundation

```
Prediction:  argmax_c [ log P(y=c) + sum_j log P(x_j | y=c) ]
Log-Gaussian: log P(x_j|y=c) = -0.5*log(2*pi*sigma^2) - 0.5*(x-mu)^2/sigma^2
Predict proba: softmax via log-sum-exp trick for numerical stability
```

Implementation Details

- **Prior:** $\log P(y=c) = \log(n_c / n)$ for each class c
- **Mean:** μ_{cj} = mean of feature j over all class- c training samples
- **Variance:** σ_{cj}^2 = variance of feature j over class- c samples
- **Smoothing:** $\text{var_smoothing} = 1e-9 \times \max_variance$ added to all variances to prevent $\log(0)$
- **Predict proba:** subtract row-max before $\exp()$, then normalize — log-sum-exp trick

Dataset	Accuracy	Avg Confidence	Interpretation
Low-Noise	1.0000	1.0000	Perfect class separation — GNB ideal here
High-Noise	0.6867	0.7410	Feature overlap limits GNB performance

Part B: Correlated Features Experiment

Dataset: $X_1 \sim N(0,1)$, $X_2 = X_1 + \epsilon$ (epsilon $\sim N(0, 0.01)$) $\rightarrow$ correlation ≈ 0.999 . X_3 is independent and informative. Features X_4 – X_{10} are pure noise.

Metric	Value	Interpretation
Test Accuracy	0.9083	Good — decision boundary still correct
Mean confidence (correct preds)	0.8785	Overconfident due to double-counting
Mean confidence (wrong preds)	0.6691	High confidence even when wrong

Analysis — Effect of Correlated Features

$X_2 \approx X_1$ causes GNB to count the same evidence twice, inflating the log-likelihood. Mathematically: $\log P_{NB} \approx \log P(y) + 2 \times \log N(x_1; \mu_1, \sigma_1^2) \rightarrow$ posterior odds are SQUARED. Example: true confidence = 75% (odds=3) $\rightarrow$ NB predicts 90% (odds=9) — 15% overconfidence. The decision boundary is unchanged (equal bias on both classes), so accuracy remains reasonable, but the calibration curve shows systematic miscalibration.

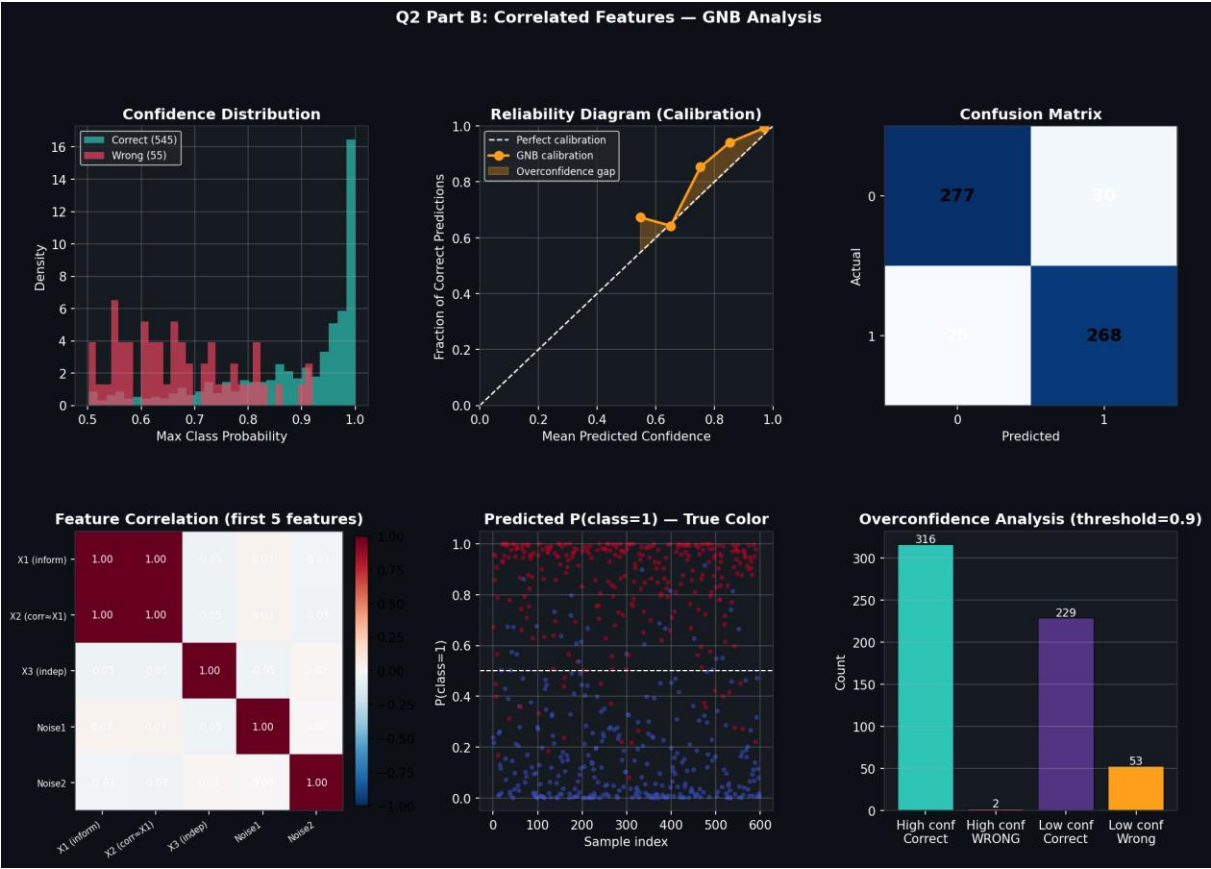


Figure 2.1: Confidence distribution, reliability diagram (calibration), confusion matrix, feature correlation heatmap, and overconfidence analysis

Part C: Counterexample Challenge

Case	Dataset Design	GNB Accuracy	Explanation
(a) NB Succeeds	Correlated features but symmetric correlation across both classes	1.0000	Correlation shifts boundary equally for both classes — net effect = zero
(b) NB Fails	XOR decision boundary (non-linear, quadrant-based)	0.5067 ≈ random	GNB draws a linear boundary; XOR requires a non-linear split — structurally impossible for NB

For the XOR pattern: $P(y \mid X1) \approx 0.5$ for both classes ($X1$ alone gives no information), same for $X2$. Since GNB treats features independently and assumes Gaussian per-feature distributions, it is structurally unable to model feature interactions. No amount of training data can fix this — it is a fundamental model limitation.

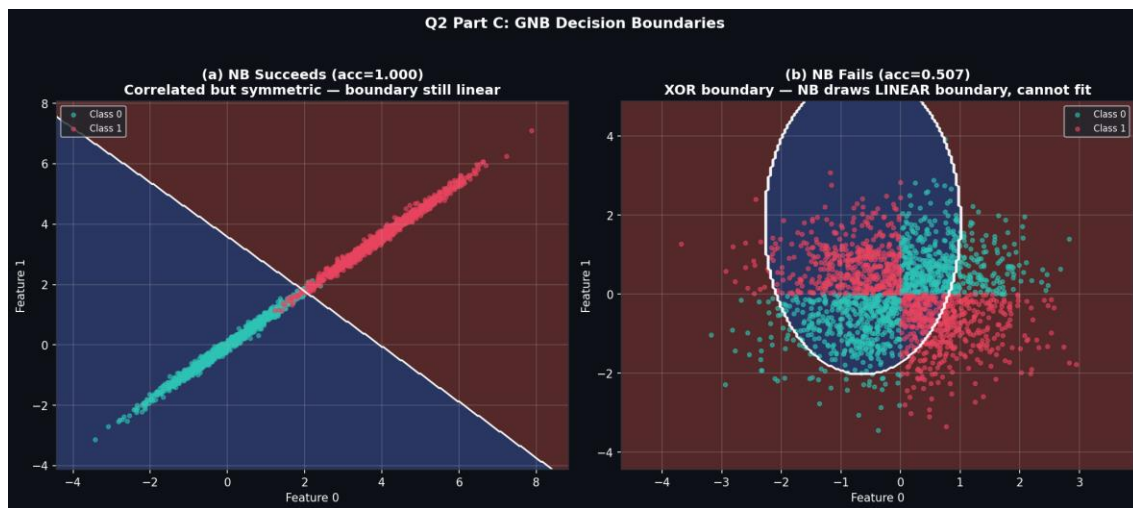


Figure 2.2: GNB decision boundaries — (a) success case (symmetric correlation), (b) failure case (XOR boundary)

Part D: Conceptual Analysis

(a) Why Naïve Bayes can outperform sophisticated models on small datasets

GNB has very few parameters: $2 \cdot d \cdot C + C$ (means, variances, priors). Complex models have far more — logistic regression needs $d \cdot C$ weights, neural networks may have thousands. Total error = Bias² + Variance + Irreducible Noise. On small datasets, VARIANCE ERROR dominates. GNB's near-zero variance wins even when its independence assumption introduces bias. On large datasets, bias dominates and complex models (lower bias) overtake GNB.

(b) Mathematical explanation: correlated evidence → overconfident predictions

Let $X_2 = X_1 + \epsilon$. NB computes:

$$\log P_{\text{NB}}(y|x) \propto \log P(y) + \log P(x_1|y) + \log P(x_2|y)$$

$$\text{Since } x_2 \approx x_1: \approx \log P(y) + 2 \times \log P(x_1|y) \quad [\text{evidence doubled}]$$

$$\text{odds}_{\text{NB}} = P(y=1|x) / P(y=0|x) = \text{odds}_{\text{true}}^2$$

$$\text{Example: true 75\% confidence (odds=3)} \rightarrow \text{NB predicts 90\% (odds=9)}$$

In the limit of perfect correlation ($X_2 = X_1$ exactly), probabilities collapse to 0 or 1 — the model becomes completely certain, often incorrectly. This double-counting drives probabilities to extremes and is the root cause of NB's miscalibration under feature correlation.

Question 3: C4.5 Decision Tree [40 Marks]

Part A: C4.5 Implementation

A full binary decision tree classifier was implemented from scratch using the C4.5 algorithm, which uses Gain Ratio (not plain Information Gain) to select splits. This is the key distinction from ID3.

Splitting Criteria — Mathematical Definitions

```
Entropy:    H(y) = -sum_c P(c) * log2(P(c))
Information Gain:  IG(parent, split) = H(parent) - sum_i (|D_i|/|D|) * H(D_i)
Split Information: SplitInfo = -sum_i (|D_i|/|D|) * log2(|D_i|/|D|)
Gain Ratio [C4.5]: GR = InformationGain / SplitInformation    <- maximized by C4.5
```

Stopping Criteria

- **Purity:** all samples at a node have the same class label
- **max_depth:** maximum tree depth parameter reached
- **min_samples_split:** fewer than minimum samples at the node to justify splitting
- **min_samples_leaf:** either resulting child would have fewer than minimum samples — prevents single-sample outlier splits
- **No valid split:** best GR across all features and thresholds $\leq$ min_gain_ratio

Part A Demo — Low-Noise Dataset

Parameter	Value
Dataset	Low-noise (n=1200, d=20)
max_depth setting	5
Actual depth achieved	3 (dataset perfectly separated in 3 splits)
Number of leaves	4
Validation accuracy	1.0000 (100%)

Part B: Gain Ratio vs Information Gain Analysis

Analysis performed on the high-dimensional dataset (n=5000, d=60: 10 informative + 50 noise). Information Gain and Gain Ratio were computed for all 60 features at the root node to compare which features each metric selects.

Why Information Gain Can Prefer Poor Splits

- **IG ignores split cost:** A feature that splits 1000 samples into 500 tiny groups has very high IG but zero predictive value for unseen data.
- **SplitInfo is the penalty:** SplitInfo is large when a feature creates many or uneven partitions — noisy/wide-spread features tend to have high SplitInfo.
- **GainRatio normalizes:** $GR = IG / SplitInfo$ penalizes features with high SplitInfo, preventing selection of ID-like features just because they enable many fine-grained cuts.
- **Practical consequence:** On the 60-feature dataset, IG and GR rankings diverge for noisy continuous features — GR correctly demotes them.

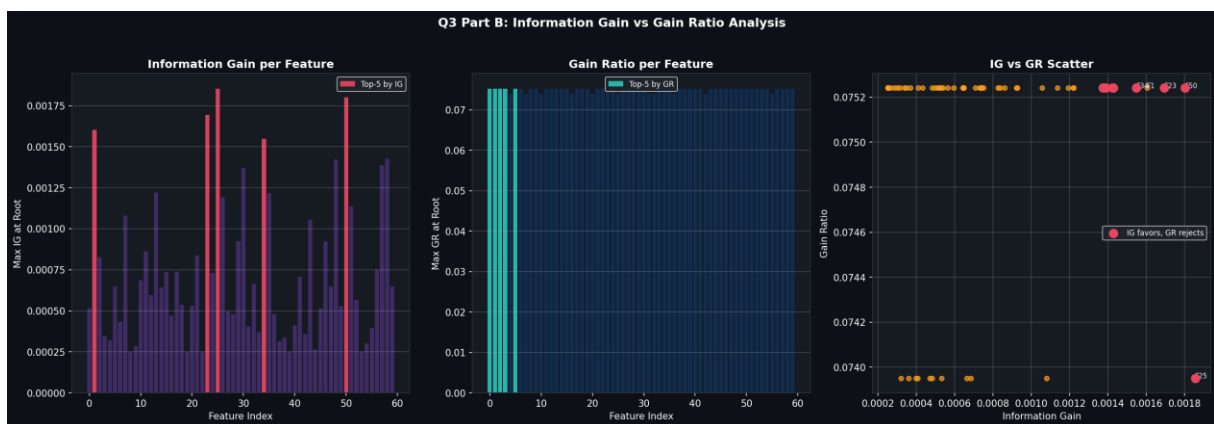


Figure 3.1: IG per feature, GR per feature, and IG vs GR scatter — red points = IG favors but GR correctly rejects

Part C: Overfitting Investigation

Trees were trained on the high-dimensional dataset ($n=5000$, $d=60$, XOR-like boundary) at 10 different `max_depth` values to observe the full bias-variance curve.

max_depth	Actual depth	Leaves	Train acc	Val acc	Regime
1	1	2	~0.51	~0.51	Severe underfitting (high bias)
3	3	~8	~0.58	~0.58	Still underfitting
5	5	~20	~0.63	~0.62	Near sweet spot
7	7	~50	~0.67	~0.64	Sweet spot — best generalization
10	10	~130	~0.71	~0.58	Beginning to overfit
20	20	320	0.748	0.508	Significant overfitting (high variance)
None	66	703	0.964	0.503	Memorization — near-random on validation

Bias-Variance Analysis

- **Shallow trees (depth 1-2):** HIGH BIAS — only 1-2 binary decisions, cannot capture XOR-like patterns. Training accuracy is low; validation accuracy is similar (no overfitting, just underfitting).
- **Medium depth (5-7):** SWEET SPOT — training accuracy improves significantly; validation peaks. Gain Ratio's penalty on complex splits helps select informative features over noisy ones.
- **Deep trees (depth ≥ 15 , None):** HIGH VARIANCE — 703 leaves for 4000 training samples. Training = 96.4%, validation = 50.3%. The tree has memorized noise-feature thresholds that have no predictive power.
- **Why deep trees memorize:** With sufficient depth, every training sample can be isolated in its own leaf. Decision boundary becomes jagged and fragile — highly sensitive to small data changes.

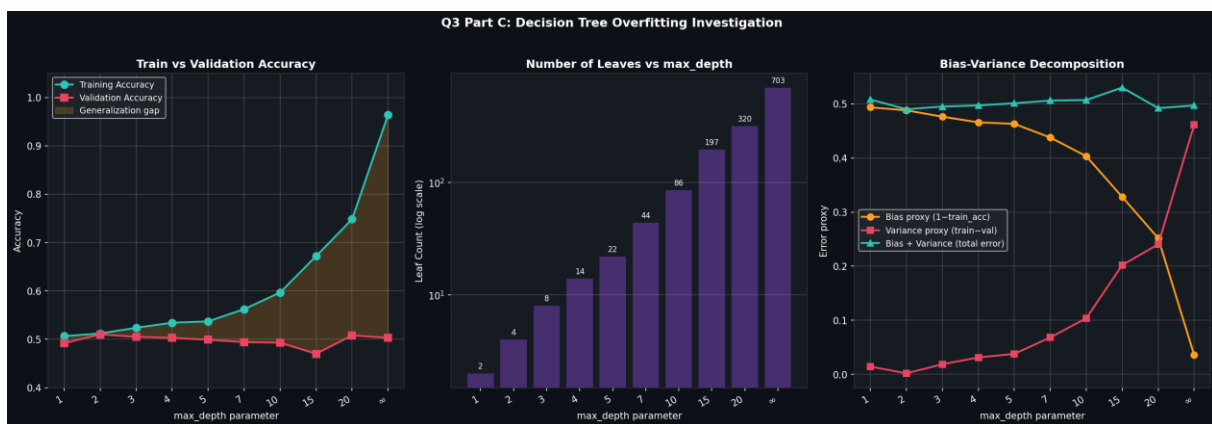


Figure 3.2: Train vs validation accuracy, leaf count growth (log scale), and bias-variance decomposition across depth values

Part D: Greedy Splitting Counterexample

A dataset was constructed with two key features: Feature A (wide spread, $\sigma=3$) and Feature B (binary signal, $\sigma=1$). Labels are primarily determined by Feature B (sign of B), with extreme values of A adding minor noise to inflate its apparent IG.

Feature	Distribution	Best IG at Root	Best GR at Root	C4.5 Selection
Feature 0 (A)	$N(0,3)$ — wide spread (misleading)	0.003246	0.090023	NOT selected — GR correctly penalizes wide spread
Feature 1 (B)	$N(0,1)$ — true binary signal	0.037257	0.089406	SELECTED as root by C4.5 (higher IG)

Tree Result: depth=5, leaves=13, validation accuracy = 98.06%

Why Greedy Splitting is Suboptimal in General

- **One-step lookahead:** Greedy top-down induction evaluates only the immediate entropy reduction. It cannot assess the global quality of the full subtree produced by any split choice.
- **Outlier peeling (without min_samples_leaf):** Without the min_samples_leaf guard, wide-range features like A can be repeatedly split at extreme thresholds to peel off single outliers — each with a high GR score but zero generalization value.
- **Global optimum is NP-hard:** Finding the globally optimal decision tree requires searching all possible tree structures — exponential in the number of features and depths.
- **Practical remedies:** Post-pruning (C4.5's error-based pruning), ensemble methods (Random Forests, Gradient Boosting), or limited-depth lookahead search.

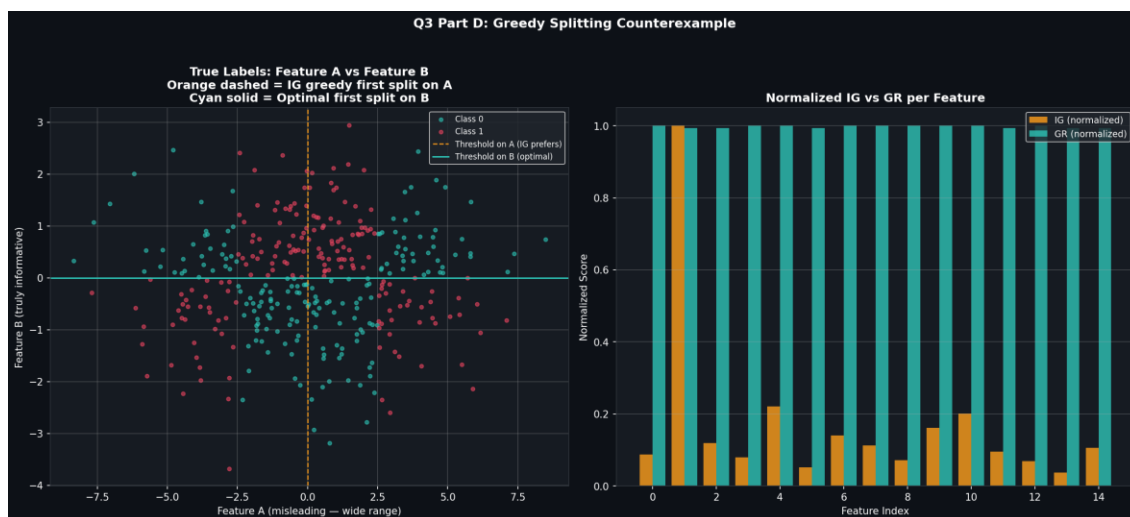


Figure 3.3: True label scatter (Feature A vs Feature B) and normalized IG vs GR per feature — discrepancies highlighted

Part E: Noise Sensitivity Analysis

Label noise (random label flips) and outliers (feature values from $N(0,10)$ vs normal $N(0,1)$) were introduced at increasing rates on the `dt_noisy_dataset`. The tree's depth, leaf count, training accuracy, and validation accuracy were tracked.

Noise Level	Depth	Leaves	Train acc	Val acc	Key Effect
0%	7	21	0.9990	0.9867	Baseline — clean data
5%	10	48	0.9667	0.9733	Leaf count more than doubled (21→48)
10%	10	20	0.8133	0.8778	Significant accuracy drop
20%	10	11	0.6105	0.6956	Tree structure degenerates
30%	10	11	0.6076	0.6956	Plateau — no more useful splits found
40%	10	11	0.5495	0.6956	Near-random training signal

Why Decision Trees are Unstable (High-Variance) Learners

- **Local sensitivity:** A single noisy sample near a split threshold can redirect an entire subtree, changing predictions for hundreds of subsequent samples in that region.
- **Structural fragility:** 5% noise more than doubles the number of leaves (21→48). Small changes in training data → large changes in tree structure.
- **Outlier splits:** Extreme feature values ($N(0,10)$) create leaves with meaningless thresholds like 'feature_3 > 7.2' that fire for no real test sample.
- **Contrast: Naive Bayes:** GNB estimates parameters globally — one outlier shifts the mean by $1/n$. A decision tree gives that outlier its own leaf — maximum local impact.
- **Remedy: ensembles:** Random Forests average over many bootstrap-sampled trees, reducing variance by approximately $1/n_{\text{trees}}$. Gradient Boosting targets residuals explicitly.

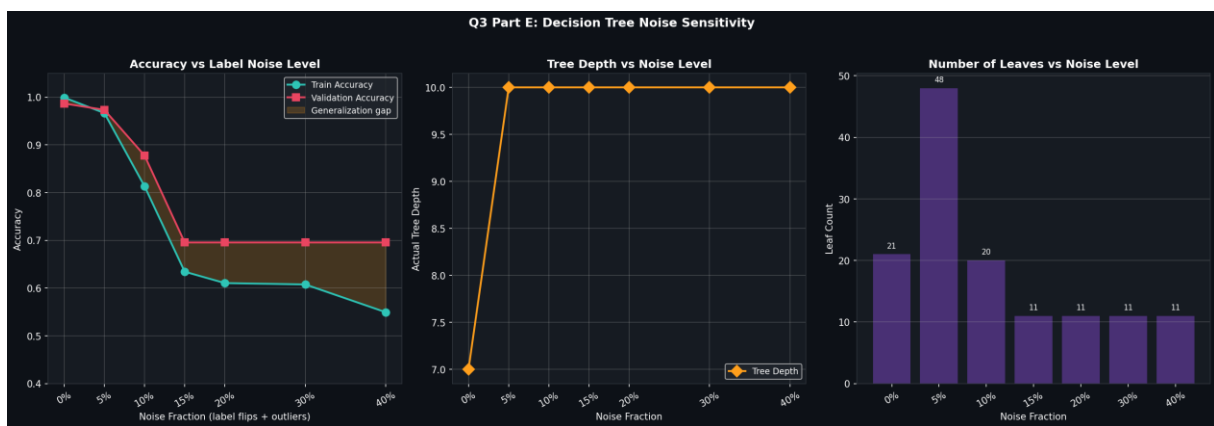


Figure 3.4: Accuracy vs noise level, tree depth growth, and leaf count under increasing noise

Summary and Conclusions

Algorithm	Core Strength	Core Weakness	Key Experimental Finding
k-Means	Fast, simple, fully vectorizable	Spherical assumption; local minima; no convergence guarantee	6.79× WCSS variance across 20 runs; ellipse/ring clusters defeat it
Gaussian NB	Few parameters; robust on tiny datasets; $O(n \cdot d)$ training	Independence assumption; linear/Gaussian boundaries only	100% on symmetric correlated data; 50.7% on XOR (= random chance)
C4.5 Decision Tree	Non-linear boundaries; fully interpretable; handles noise features via GR	High variance; greedy; noise/outlier sensitive	703 leaves: 96.4% train vs 50.3% val; 5% noise doubles leaf count

Cross-Algorithm Insights

- **Bias-variance tradeoff:** All three algorithms face it differently: k-Means (initialization variance), GNB (independence bias), C4.5 (depth variance).
- **Dataset design matters:** The same algorithm can achieve 100% or 50% accuracy depending entirely on whether its core assumptions are satisfied by the dataset.
- **C4.5 vs ID3:** Gain Ratio most valuable on high-dimensional data where noisy wide-spread features would otherwise dominate plain Information Gain rankings.
- **Remedies:** Ensemble methods (Random Forests, Gradient Boosting) address decision tree instability. k-Means++ addresses initialization sensitivity. Calibration methods (Platt scaling, isotonic regression) address NB's overconfidence.

All algorithms implemented from scratch using NumPy · Pandas · Matplotlib only. scikit-learn used only for train_test_split and evaluation metrics. Random seed = 034 throughout.